Tarea 2: Procedimientos y clausuras

Angel

Parte 1

1.1 En el nucleo

Con $es = e_1 \cdots e_n$ $(n \geq 0)$:

$$\frac{}{(\texttt{value-of}\ (\texttt{proc-exp}\ xs\ c)\ \rho) = \mathsf{clausura}(xs, c, \rho)}\ [\textsc{proc}]$$

$$\frac{(\texttt{value-of}\ e_0\ \rho) = \mathsf{clausura}(xs, c, \rho') \qquad (\texttt{value-of}\ e_i\ \rho) = v_i \quad (1 \leq i \leq n) \qquad |xs| = n}{(\texttt{value-of}\ (\texttt{call-exp}\ e_0\ es)\ \rho) = (\texttt{value-of}\ c\ [xs{=}v_1 \cdots v_n]\rho')}\ [\textsc{call}]$$

$$\frac{\begin{array}{c}(\texttt{value-of}\ e_0\ \rho) = \mathsf{clausura}(xs, c, \rho') \\ (\texttt{value-of}\ e_i\ \rho) = v_i \quad (1 \leq i \leq n) \qquad |xs| = m \neq n\end{array}}{(\texttt{value-of}\ (\texttt{call-exp}\ e_0\ es)\ \rho) = \texttt{error: esperaba}\ m\ \texttt{y me diste}\ n}\ [\textsc{call-aridad}]$$

1.2 Como azucar

$$\texttt{proc}\ (x_1,\ x_2)\ c \rightsquigarrow \texttt{proc}\ (x_1)\ \texttt{proc}\ (x_2)\ c \qquad [\text{proc*}]$$

$$\texttt{proc}\ (x_1,\ x_2,\ \ldots,\ x_n)\ c \rightsquigarrow \texttt{proc}\ (x_1)\ \texttt{proc}\ (x_2,\ \ldots,\ x_n)\ c \qquad (n \geq 3)$$

$$(e_0\ e_1\ e_2) \rightsquigarrow ((e_0\ e_1)\ e_2) \qquad [\text{call*}]$$

$$(e_0\ e_1\ e_2\ \ldots\ e_n) \rightsquigarrow ((e_0\ e_1)\ e_2\ \ldots\ e_n) \qquad (n \geq 3)$$

1.3 Comparacion

	Nucleo	**Azucar**
P1	`7`	`7`
P2	`apply-procedure: numero de argumentos equivocado: esperaba 2 y me diste 1`	`7`
P3	`apply-procedure: numero de argumentos equivocado: esperaba 2 y me diste 3`	`val->proc: Esperaba un procedimiento pero me diste 7`
P4	`5`	`parse: no esperaba ‘)’ despues de ‘(’`
P5	`apply-procedure: numero de argumentos equivocado: esperaba 1 y me diste 2`	`7`

Nucleo. Habilita procedimientos sin parametros (P4) y errores de aridad claros en la misma llamada (P3). Sacrifica la aplicacion parcial (P2) y ya no se puede llamar con varios operandos a un procedimiento currificado (P5).

Azucar. Habilita la aplicacion parcial (P2) y llamar igual a procedimientos currificados y de varios parametros (P1, P5), sin tocar el interprete. Sacrifica los procedimientos sin parametros (P4) y el error de aridad: un operando de mas truena despues y con otro mensaje (P3).

1. Que es `(resta 10)`? En el nucleo es un error de aridad, porque `resta` espera 2 argumentos (P2). En el azucar `resta` es `proc (x) proc (y) -(x, y)`, entonces `(resta 10)` es un procedimiento que espera la `y`, con $x = 10$.

2. Cuando se detecta un operando de mas? En el nucleo en la misma llamada, antes de evaluar el cuerpo: `esperaba 2 y me diste 3` (P3). En el azucar `(resta 10 3 1)` se vuelve `(((resta 10) 3) 1)`, las dos primeras llamadas dan `7` y el error sale al intentar llamar a `7`: `val->proc: Esperaba un procedimiento pero me diste 7`, que no dice nada de argumentos.

3. Que escribe el REPL para `proc (x, y) x`? En el nucleo `<proc (x, y)>`, porque la clausura guarda la lista de parametros. En el azucar `<proc (x)>`, porque `desugar` lo traduce a `proc (x) proc (y) x` antes de evaluar, y el valor es la clausura de afuera, que solo tiene `x`.

Que modifique. En el nucleo: `ast.rkt`, `parser.rkt`, `desugar.rkt`, `procs.rkt`, `vals.rkt` e `interp.rkt` (`value-of` de `proc-exp` y `call-exp`, y `apply-procedure`). En el azucar solo `ast.rkt`, `parser.rkt` y `desugar.rkt`; el interprete no se toca.

Parte 2

2.1 Especificacion

$$\frac{}{(\texttt{value-of}\ (\texttt{proc-exp}\ x\ c)\ \rho) = \mathsf{procedimiento}(x, c)}\ [\textsc{proc}]$$

$$\frac{(\texttt{value-of}\ e_1\ \rho) = \mathsf{procedimiento}(x, c) \qquad (\texttt{value-of}\ e_2\ \rho) = v_2}{(\texttt{value-of}\ (\texttt{call-exp}\ e_1\ e_2)\ \rho) = (\texttt{value-of}\ c\ [x{=}v_2]\rho)}\ [\textsc{call}]$$

Cambia la conclusion de [call]: el cuerpo se evalua en $[x{=}v_2]\rho$, el entorno de la llamada, en vez de $[x{=}v_2]\rho'$. Como ρ' ya no se usa en ningun lado, el valor de un `proc` ya no necesita guardar un entorno, solo el parametro y el cuerpo.

2.3 Analisis

	PROC (lexico)	**proc-dyn (dinamico)**
D1	`3`	`5`
D2	`3`	`2`
D3	`-100`	`0`
D4	`7`	`apply-env: la variable n no esta vinculada`
D5	`apply-env: la variable y no esta vinculada`	`5`

Lexico. Habilita que un procedimiento devuelva otro que recuerde sus variables (D4) y que el resultado de un procedimiento no dependa de desde donde se llama ni de los nombres de quien lo llama (D1 y D2 dan lo mismo, en D3 `f` y `g` usan cada uno su `x`). Sacrifica poder usar variables que todavia no existen cuando se crea el procedimiento, aunque si existan cuando se llama (D5).

Dinamico. Habilita que un procedimiento use variables de quien lo llama sin pasarlas como argumento (D5). Sacrifica los procedimientos que devuelven procedimientos (D4) y que el resultado dependa solo del codigo del procedimiento: depende de los nombres que haya donde se llama (D1, D2, D3).

1. Por que en D1 y D2 cambiar el nombre del parametro cambia el resultado? Con vinculacion dinamica la `a` del cuerpo de `p` se busca en el entorno de la llamada a `p`, que esta dentro de `f`. En D1 ese entorno tiene `a = 5` (el `let` de afuera), y en D2 el parametro `a = 2` de `f` la tapa. En PROC la `a` de `p` se busca en el entorno donde se creo `p`, donde `a = 3`, y el nombre del parametro de `f` no importa. Quien escribe `f` tiene que saber que variables libres usan los procedimientos que llama, para no taparlas con sus propios nombres, y tiene que saber que hay vinculado en cada lugar desde donde se va a llamar.

2. Que le pasa a un procedimiento que devuelve otro (D4)? En PROC `(menos 5)` devuelve una clausura que guarda `n = 5`, y al llamarla con `12` da `7`. Con vinculacion dinamica el procedimiento devuelto no guarda entorno, y cuando se llama, `n` ya no existe en el entorno de la llamada, porque solo existia durante la llamada a `menos`: da error.

3. Por que el alcance lexico es la norma? Porque con alcance lexico se puede saber a que se refiere cada variable leyendo solo el codigo donde esta escrita, sin conocer a quien llama. Consecuencias concretas: renombrar un parametro nunca cambia el resultado (D1 y D2), y la currificacion funciona (D4). Sin esto, la parte 1.2 no serviria: `proc (x, y)` se traduce a procedimientos anidados, y `(resta 10)` perderia la `x` igual que `menos` pierde la `n`.